

Appendix C: LeNet5 Test

In [1]: `!nvidia-smi`

```
Fri Mar 14 01:34:47 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                       |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off |  00000000:00:04.0 Off |
0|
| N/A    29C    P0              45W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                       |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import ResNet50 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis of model
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Load data

```

In [4]: # Load CIFAR-100 dataset
(X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

# Split (8000) of training data into temporary set
X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=0.1)
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_temp, test_size=0.5,
                             random_state=42)
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5,
                                                random_state=42)

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a 70/30 split
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}")

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 3s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

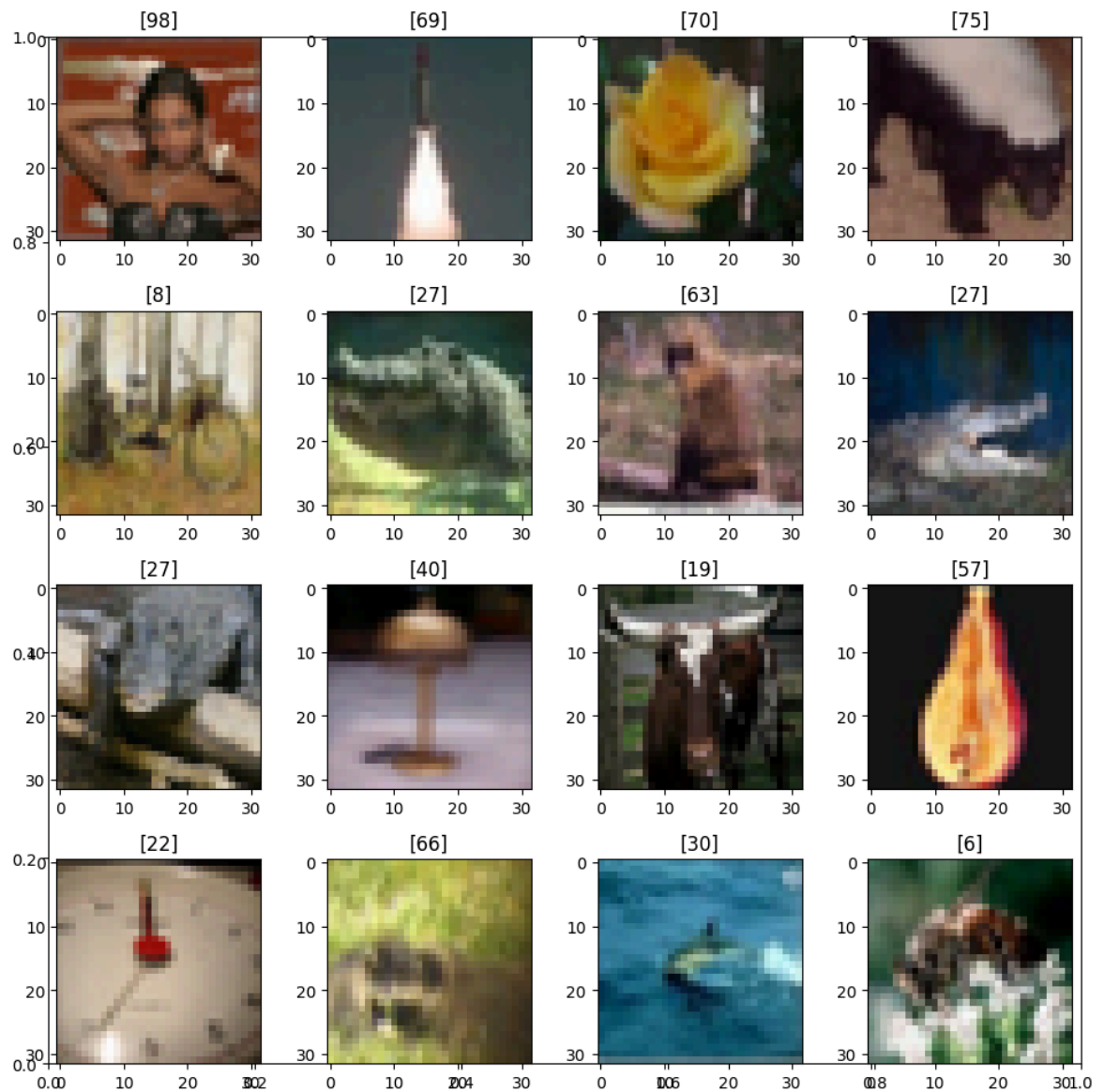
y_test.shape: (9000, 1)

```
In [5]: idxs_disp = []
def display_imgs(imgs, labels, norm_version):
    plt.subplots(figsize=(10,10))
    if norm_version == 0:
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            idxs_disp.append(k)
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.title(labels[k])
            plt.imshow(imgs[k])
    else:
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = idxs_disp[i]
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.title(labels[k])
            plt.imshow(imgs[k])
    plt.tight_layout()
    plt.show()

display_imgs(X_train, y_train, 0)
```

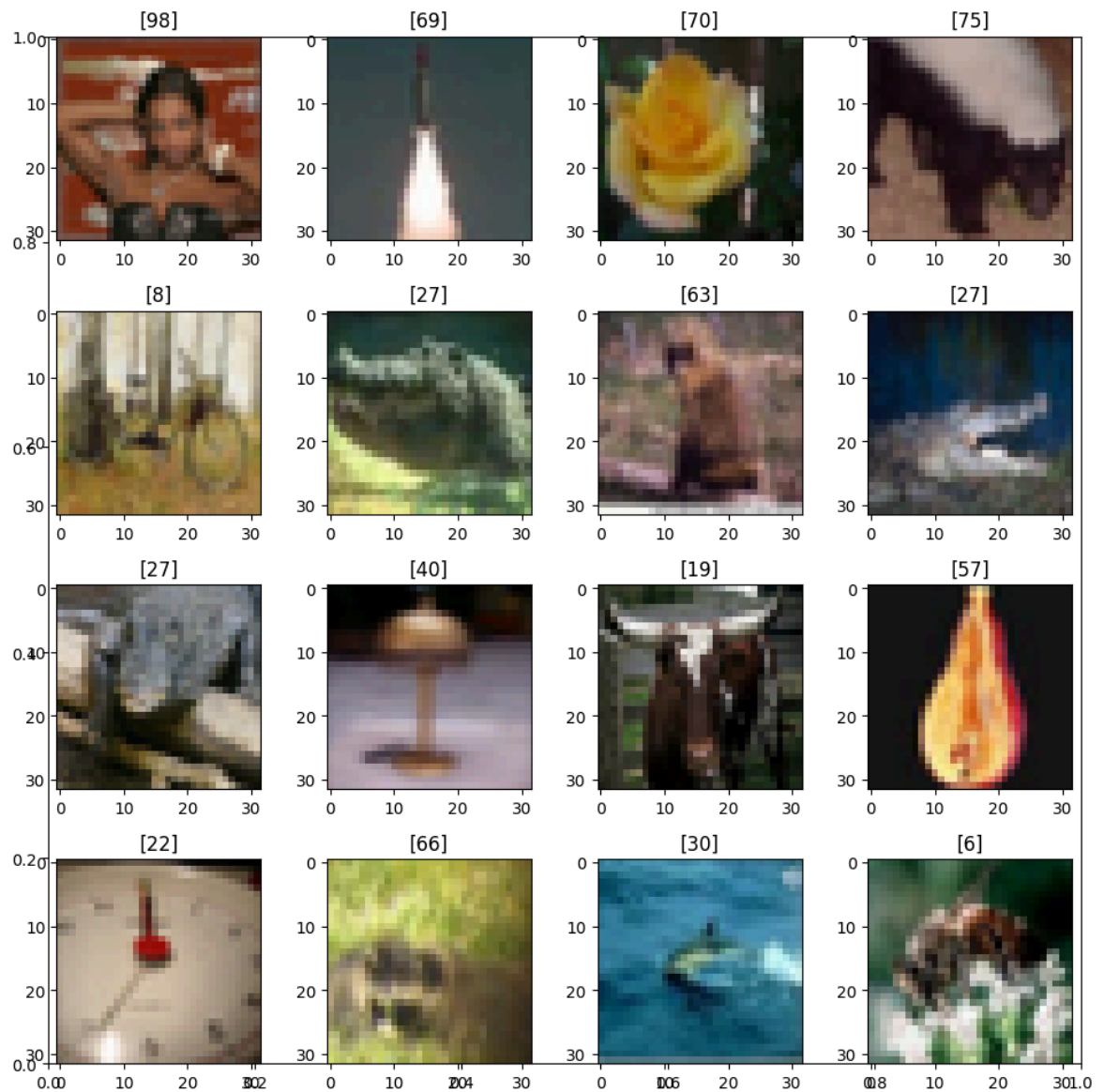
```
labels[17107].shape: (1,)
```

```
imgs[17107].shape: (32, 32, 3)
```



```
In [6]: # Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train, 1)
```

```
labels[17107].shape: (1,)
imgs[17107].shape: (32, 32, 3)
```



```
In [7]: #train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)).shuffle(
#val_dataset = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(64)
#test_dataset = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(64)

labels_names = ['beaver', 'dolphins', 'otter', 'seal', 'whale', 'aquarium fish', 'flatfish',
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bowls',
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock', 'cups',
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardrobe',
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wolf',
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea', 'cat',
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 'skunk',
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile', 'dinosaur',
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple', 'cactus',
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mower',
'tractor']

# Create TensorFlow datasets (may need imgs as 224,224 Later on)

batch_size = 64
train_dataset = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(lambda x, y: (tf.image.resize(x, (32, 32)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.int32))),
                .batch(batch_size))
```

```

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (32, 32)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))
               .batch(batch_size))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (32, 32)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))
               .batch(batch_size))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 32, 32, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 32, 32, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 32, 32, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

```

Training dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 32, 32, 3)
Label shape: (64, 100)

```

Validation dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 32, 32, 3)
Label shape: (64, 100)

```

Testing dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 32, 32, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 32, 32, 3)
Label shape: (64, 100)

```

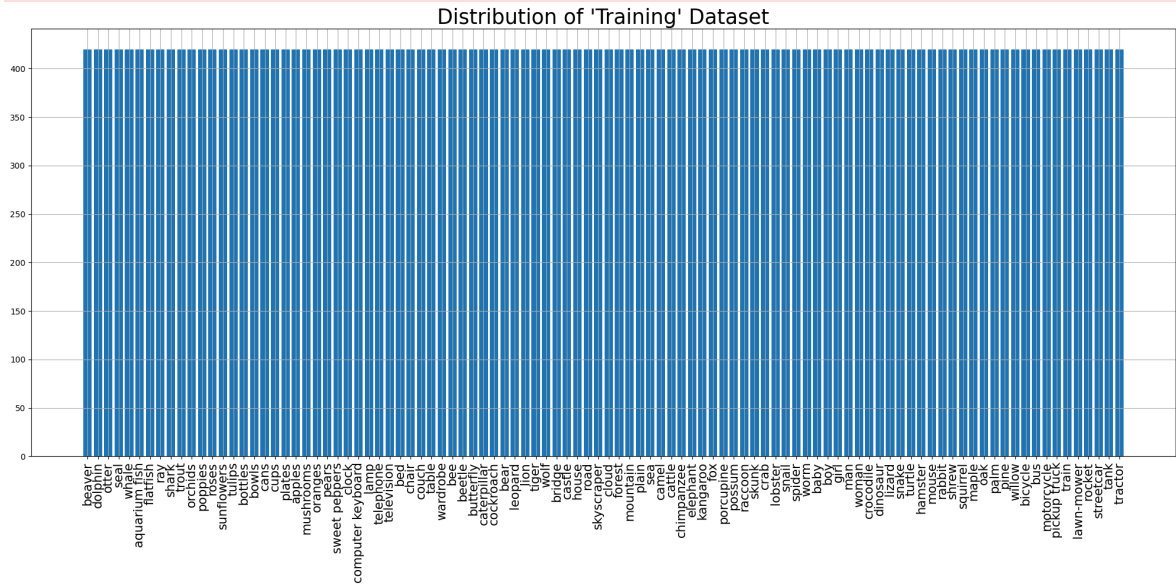
```

In [8]: def class_distrib(y, labels_names, dataset_name):
        counts = pd.DataFrame(data=y).value_counts().sort_index()
        #print(f"counts:\n{counts}")
        fig, ax = plt.subplots(figsize=(20,10))
        ax.bar(labels_names, counts)
        ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
        plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
        plt.grid()
        plt.tight_layout()
        plt.show()

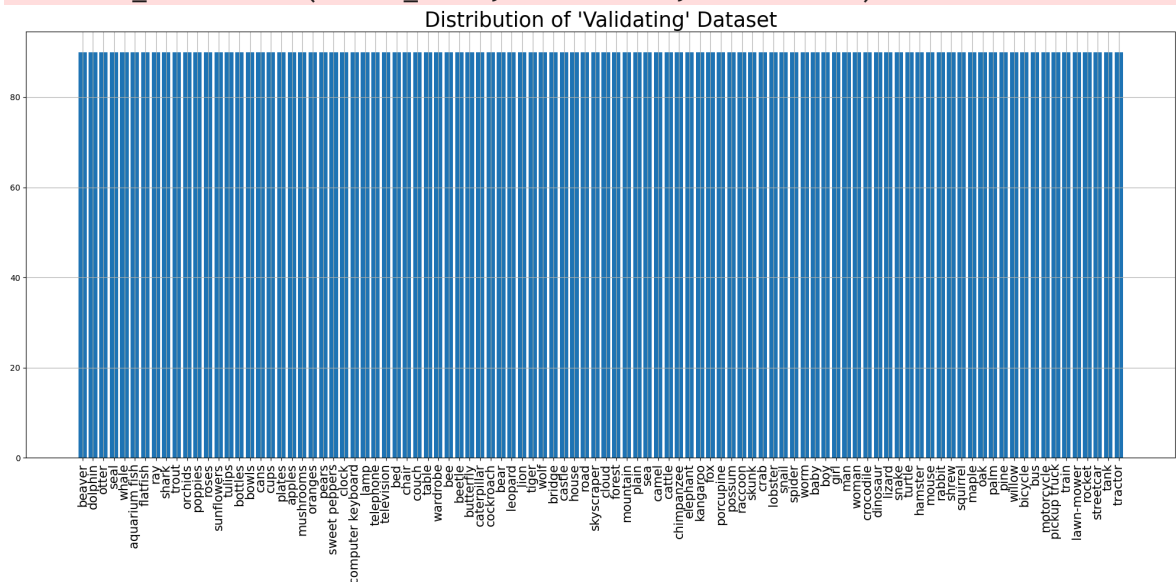
        class_distrib(y_train, labels_names, "Training")
        class_distrib(y_val, labels_names, "Validating")
        class_distrib(y_test, labels_names, "Testing")

```

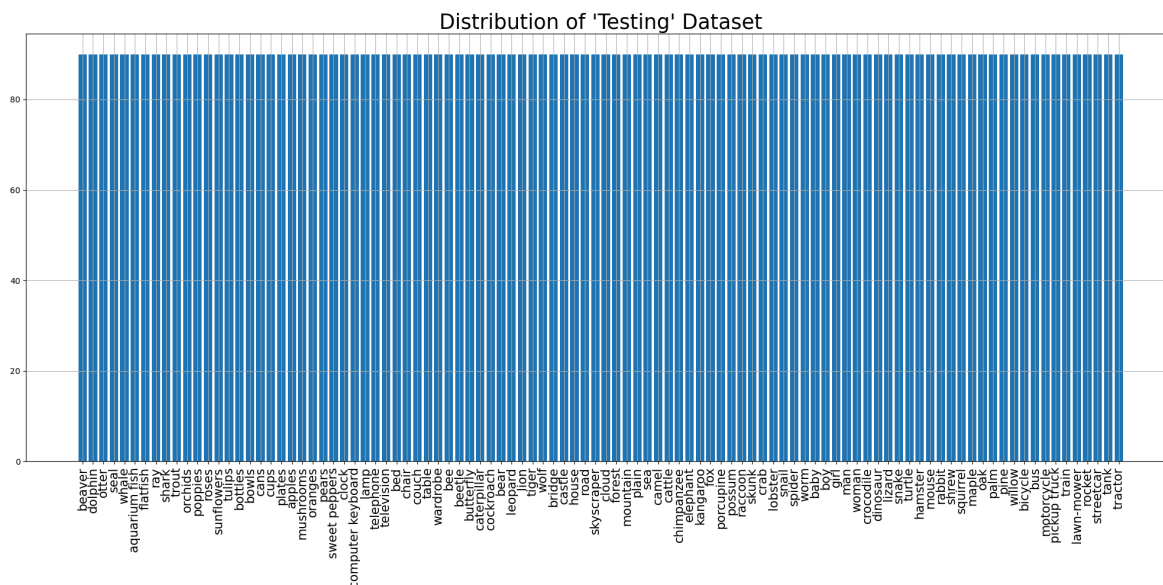
```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Define Model

```
In [10]: def build_lenet():
# Let-Net5 Conv-net architecture
model = models.Sequential([
    layers.Conv2D(6, kernel_size=(5, 5), activation='tanh', padding="same",
    layers.AveragePooling2D(pool_size=(2, 2), strides=(2,2)),

    layers.Conv2D(16, kernel_size=(5, 5), activation='tanh'),
    layers.AveragePooling2D(pool_size=(2, 2), strides=(2,2)),

    layers.Flatten(),
    layers.Dense(120, activation='tanh'),
    layers.Dense(84, activation='tanh'),
    layers.Dense(100, activation='softmax') # 100 classes
])

model.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy', # Or sparse?
              metrics=['accuracy'])

return model

# Build the model
model = build_lenet()
model.summary()
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 32, 32, 6)
average_pooling2d (AveragePooling2D)	(None, 16, 16, 6)
conv2d_1 (Conv2D)	(None, 12, 12, 16)
average_pooling2d_1 (AveragePooling2D)	(None, 6, 6, 16)
flatten (Flatten)	(None, 576)
dense (Dense)	(None, 120)
dense_1 (Dense)	(None, 84)
dense_2 (Dense)	(None, 100)



Total params: 90,776 (354.59 KB)

Trainable params: 90,776 (354.59 KB)

Non-trainable params: 0 (0.00 B)

Train Model & Track Training/Validation Error

```
In [11]: # reduceLR = ReduceLRonPateau(monitor='val_accuracy', factor=0.1, patience=3, mi

# Train model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=20, verbo
```

Epoch 1/20
657/657 ————— 22s 16ms/step - accuracy: 0.0617 - loss: 4.2293 - val_accuracy: 0.1410 - val_loss: 3.7386

Epoch 2/20
657/657 ————— 2s 3ms/step - accuracy: 0.1573 - loss: 3.6542 - val_accuracy: 0.1753 - val_loss: 3.5585

Epoch 3/20
657/657 ————— 2s 3ms/step - accuracy: 0.1859 - loss: 3.4683 - val_accuracy: 0.1962 - val_loss: 3.4519

Epoch 4/20
657/657 ————— 2s 3ms/step - accuracy: 0.2097 - loss: 3.3348 - val_accuracy: 0.2100 - val_loss: 3.3773

Epoch 5/20
657/657 ————— 2s 3ms/step - accuracy: 0.2281 - loss: 3.2263 - val_accuracy: 0.2163 - val_loss: 3.3191

Epoch 6/20
657/657 ————— 2s 3ms/step - accuracy: 0.2445 - loss: 3.1378 - val_accuracy: 0.2200 - val_loss: 3.2781

Epoch 7/20
657/657 ————— 2s 3ms/step - accuracy: 0.2589 - loss: 3.0632 - val_accuracy: 0.2262 - val_loss: 3.2543

Epoch 8/20
657/657 ————— 2s 3ms/step - accuracy: 0.2676 - loss: 2.9991 - val_accuracy: 0.2289 - val_loss: 3.2498

Epoch 9/20
657/657 ————— 2s 3ms/step - accuracy: 0.2773 - loss: 2.9434 - val_accuracy: 0.2310 - val_loss: 3.2492

Epoch 10/20
657/657 ————— 2s 3ms/step - accuracy: 0.2870 - loss: 2.8933 - val_accuracy: 0.2357 - val_loss: 3.2503

Epoch 11/20
657/657 ————— 2s 3ms/step - accuracy: 0.2932 - loss: 2.8442 - val_accuracy: 0.2341 - val_loss: 3.2563

Epoch 12/20
657/657 ————— 2s 3ms/step - accuracy: 0.3011 - loss: 2.7990 - val_accuracy: 0.2308 - val_loss: 3.2659

Epoch 13/20
657/657 ————— 2s 3ms/step - accuracy: 0.3106 - loss: 2.7585 - val_accuracy: 0.2307 - val_loss: 3.2780

Epoch 14/20
657/657 ————— 2s 3ms/step - accuracy: 0.3192 - loss: 2.7180 - val_accuracy: 0.2311 - val_loss: 3.2912

Epoch 15/20
657/657 ————— 2s 3ms/step - accuracy: 0.3278 - loss: 2.6811 - val_accuracy: 0.2299 - val_loss: 3.3061

Epoch 16/20
657/657 ————— 2s 3ms/step - accuracy: 0.3364 - loss: 2.6505 - val_accuracy: 0.2281 - val_loss: 3.3075

Epoch 17/20
657/657 ————— 2s 3ms/step - accuracy: 0.3419 - loss: 2.6299 - val_accuracy: 0.2274 - val_loss: 3.3238

Epoch 18/20
657/657 ————— 2s 3ms/step - accuracy: 0.3445 - loss: 2.6162 - val_accuracy: 0.2248 - val_loss: 3.3444

Epoch 19/20
657/657 ————— 2s 3ms/step - accuracy: 0.3490 - loss: 2.5935 - val_accuracy: 0.2243 - val_loss: 3.3615

Epoch 20/20
657/657 ————— 2s 3ms/step - accuracy: 0.3519 - loss: 2.5687 - val_accuracy: 0.2224 - val_loss: 3.3873

Plot Training & Validation Error

```
In [12]: # Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

    # Plot Training and Validation Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.legend()
    plt.grid()

    plt.tight_layout()
    plt.show()

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)
```



Evaluate Model on Test Data

```
In [13]: # Evaluate on test data
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test Accuracy: {test_acc*100:.2f}%")
```

141/141 ————— 0s 2ms/step - accuracy: 0.2256 - loss: 3.3893
Test Accuracy: 22.07%

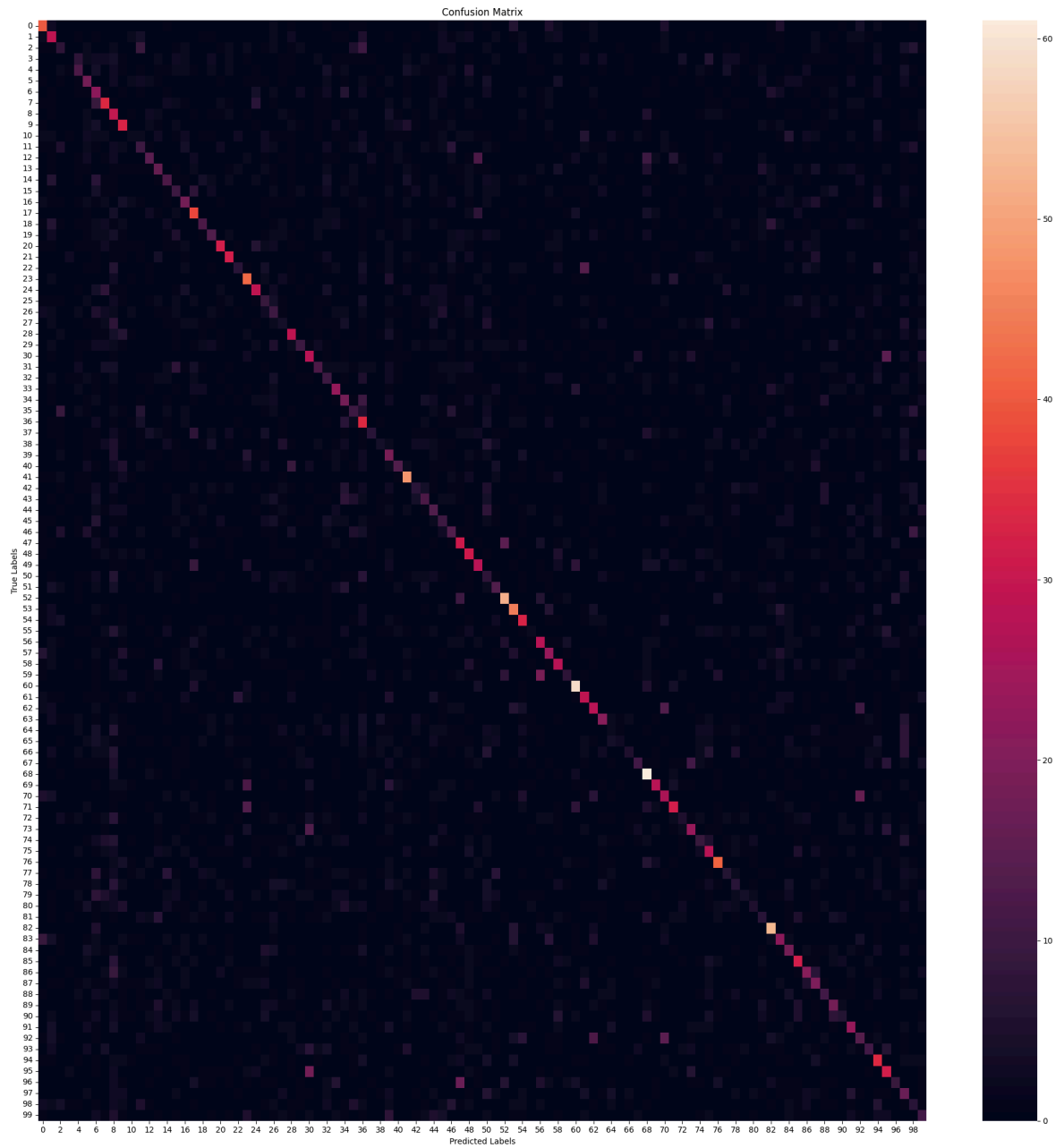
Generate Confusion Matrix

```
In [14]: # Get predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()
```

282/282 ————— 4s 7ms/step



```
In [15]: print(classification_report(y_true_classes, y_pred_classes, target_names=labels_
```

	precision	recall	f1-score	support
beaver	0.41	0.44	0.43	90
dolphin	0.21	0.32	0.26	90
otter	0.12	0.09	0.10	90
seal	0.00	0.00	0.00	90
whale	0.16	0.13	0.15	90
aquarium fish	0.19	0.20	0.19	90
flatfish	0.12	0.24	0.16	90
ray	0.28	0.38	0.32	90
shark	0.11	0.33	0.17	90
trout	0.26	0.37	0.31	90
orchids	0.06	0.02	0.03	90
poppies	0.13	0.11	0.12	90
roses	0.24	0.17	0.20	90
sunflowers	0.17	0.18	0.17	90
tulips	0.17	0.13	0.15	90
bottles	0.11	0.11	0.11	90
bowls	0.21	0.20	0.21	90
cans	0.29	0.42	0.34	90
cups	0.21	0.13	0.16	90
plates	0.20	0.14	0.17	90
apples	0.46	0.36	0.40	90
mushrooms	0.36	0.36	0.36	90
oranges	0.20	0.08	0.11	90
pears	0.33	0.47	0.38	90
sweet peppers	0.37	0.32	0.34	90
clock	0.10	0.10	0.10	90
computer keyboard	0.09	0.11	0.10	90
lamp	0.06	0.04	0.05	90
telephone	0.27	0.32	0.29	90
television	0.20	0.11	0.14	90
bed	0.26	0.31	0.29	90
chair	0.22	0.13	0.17	90
couch	0.09	0.11	0.10	90
table	0.26	0.26	0.26	90
wardrobe	0.15	0.20	0.17	90
bee	0.15	0.11	0.13	90
beetle	0.17	0.38	0.24	90
butterfly	0.21	0.08	0.11	90
caterpillar	0.12	0.04	0.06	90
cockroach	0.13	0.21	0.16	90
bear	0.25	0.14	0.18	90
leopard	0.38	0.53	0.44	90
lion	0.12	0.08	0.10	90
tiger	0.17	0.13	0.15	90
wolf	0.11	0.14	0.12	90
bridge	0.10	0.11	0.11	90
castle	0.12	0.14	0.13	90
house	0.26	0.34	0.30	90
road	0.21	0.34	0.26	90
skyscraper	0.31	0.31	0.31	90
cloud	0.06	0.09	0.07	90
forest	0.17	0.14	0.15	90
mountain	0.56	0.58	0.57	90
plain	0.41	0.50	0.45	90
sea	0.27	0.37	0.31	90
camel	0.09	0.02	0.04	90
cattle	0.31	0.31	0.31	90
chimpanzee	0.28	0.26	0.27	90

elephant	0.25	0.31	0.28	90
kangaroo	0.25	0.09	0.13	90
fox	0.54	0.66	0.59	90
porcupine	0.27	0.32	0.30	90
possum	0.31	0.31	0.31	90
raccoon	0.27	0.23	0.25	90
skunk	0.04	0.02	0.03	90
crab	0.23	0.03	0.06	90
lobster	0.19	0.06	0.09	90
snail	0.30	0.12	0.17	90
spider	0.38	0.68	0.48	90
worm	0.33	0.31	0.32	90
baby	0.27	0.29	0.28	90
boy	0.38	0.36	0.37	90
girl	0.12	0.06	0.08	90
man	0.34	0.26	0.29	90
woman	0.15	0.10	0.12	90
crocodile	0.22	0.31	0.26	90
dinosaur	0.42	0.46	0.44	90
lizard	0.11	0.07	0.08	90
snake	0.09	0.08	0.08	90
turtle	0.14	0.04	0.07	90
hamster	0.12	0.06	0.08	90
mouse	0.09	0.08	0.08	90
rabbit	0.42	0.59	0.49	90
shrew	0.23	0.24	0.24	90
squirrel	0.15	0.20	0.17	90
maple	0.32	0.36	0.34	90
oak	0.24	0.23	0.24	90
palm	0.18	0.22	0.20	90
pine	0.15	0.12	0.13	90
willow	0.23	0.20	0.21	90
bicycle	0.15	0.07	0.09	90
bus	0.21	0.26	0.23	90
motorcycle	0.14	0.16	0.15	90
pickup truck	0.22	0.12	0.16	90
train	0.20	0.38	0.26	90
lawn-mower	0.30	0.36	0.32	90
rocket	0.28	0.10	0.15	90
streetcar	0.09	0.19	0.13	90
tank	0.08	0.08	0.08	90
tractor	0.10	0.12	0.11	90
accuracy			0.22	9000
macro avg	0.22	0.22	0.21	9000
weighted avg	0.22	0.22	0.21	9000

```
In [16]: for xxxx in range(5):
# reduceLR = ReduceLRonPateau(monitor='val_accuracy', factor=0.1, patience=3

# Train model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=20, v

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
```

```
plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

# Evaluate on test data
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test Accuracy: {test_acc*100:.2f}%")

# Get predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

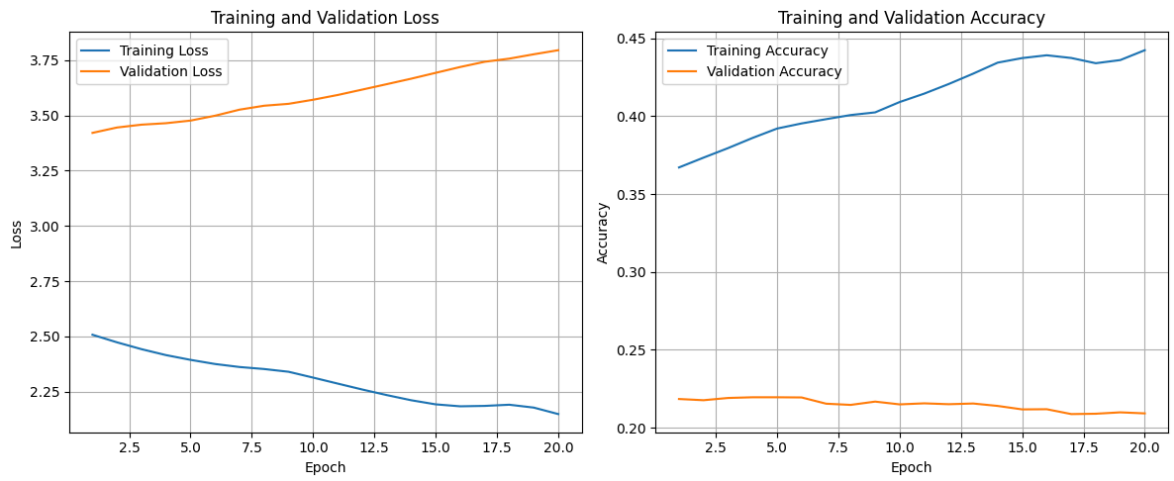
print(classification_report(y_true_classes, y_pred_classes, target_names=lab
```



```

Epoch 1/20
657/657  2s 3ms/step - accuracy: 0.3563 - loss: 2.5433 - val_
accuracy: 0.2184 - val_loss: 3.4207
Epoch 2/20
657/657  2s 3ms/step - accuracy: 0.3612 - loss: 2.5134 - val_
accuracy: 0.2177 - val_loss: 3.4451
Epoch 3/20
657/657  2s 3ms/step - accuracy: 0.3670 - loss: 2.4863 - val_
accuracy: 0.2191 - val_loss: 3.4579
Epoch 4/20
657/657  2s 3ms/step - accuracy: 0.3719 - loss: 2.4620 - val_
accuracy: 0.2196 - val_loss: 3.4645
Epoch 5/20
657/657  2s 3ms/step - accuracy: 0.3792 - loss: 2.4418 - val_
accuracy: 0.2196 - val_loss: 3.4764
Epoch 6/20
657/657  2s 3ms/step - accuracy: 0.3834 - loss: 2.4201 - val_
accuracy: 0.2194 - val_loss: 3.4984
Epoch 7/20
657/657  2s 3ms/step - accuracy: 0.3883 - loss: 2.3988 - val_
accuracy: 0.2154 - val_loss: 3.5259
Epoch 8/20
657/657  2s 3ms/step - accuracy: 0.3930 - loss: 2.3772 - val_
accuracy: 0.2147 - val_loss: 3.5436
Epoch 9/20
657/657  2s 3ms/step - accuracy: 0.3977 - loss: 2.3535 - val_
accuracy: 0.2168 - val_loss: 3.5522
Epoch 10/20
657/657  2s 3ms/step - accuracy: 0.4061 - loss: 2.3287 - val_
accuracy: 0.2150 - val_loss: 3.5707
Epoch 11/20
657/657  2s 3ms/step - accuracy: 0.4100 - loss: 2.3015 - val_
accuracy: 0.2157 - val_loss: 3.5921
Epoch 12/20
657/657  2s 3ms/step - accuracy: 0.4181 - loss: 2.2731 - val_
accuracy: 0.2151 - val_loss: 3.6165
Epoch 13/20
657/657  2s 3ms/step - accuracy: 0.4236 - loss: 2.2461 - val_
accuracy: 0.2156 - val_loss: 3.6410
Epoch 14/20
657/657  2s 3ms/step - accuracy: 0.4333 - loss: 2.2211 - val_
accuracy: 0.2140 - val_loss: 3.6659
Epoch 15/20
657/657  2s 3ms/step - accuracy: 0.4377 - loss: 2.2015 - val_
accuracy: 0.2118 - val_loss: 3.6923
Epoch 16/20
657/657  2s 3ms/step - accuracy: 0.4388 - loss: 2.1896 - val_
accuracy: 0.2119 - val_loss: 3.7188
Epoch 17/20
657/657  2s 3ms/step - accuracy: 0.4381 - loss: 2.1881 - val_
accuracy: 0.2088 - val_loss: 3.7424
Epoch 18/20
657/657  2s 3ms/step - accuracy: 0.4325 - loss: 2.1963 - val_
accuracy: 0.2090 - val_loss: 3.7566
Epoch 19/20
657/657  2s 3ms/step - accuracy: 0.4327 - loss: 2.1975 - val_
accuracy: 0.2099 - val_loss: 3.7763
Epoch 20/20
657/657  2s 3ms/step - accuracy: 0.4359 - loss: 2.1726 - val_
accuracy: 0.2092 - val_loss: 3.7948

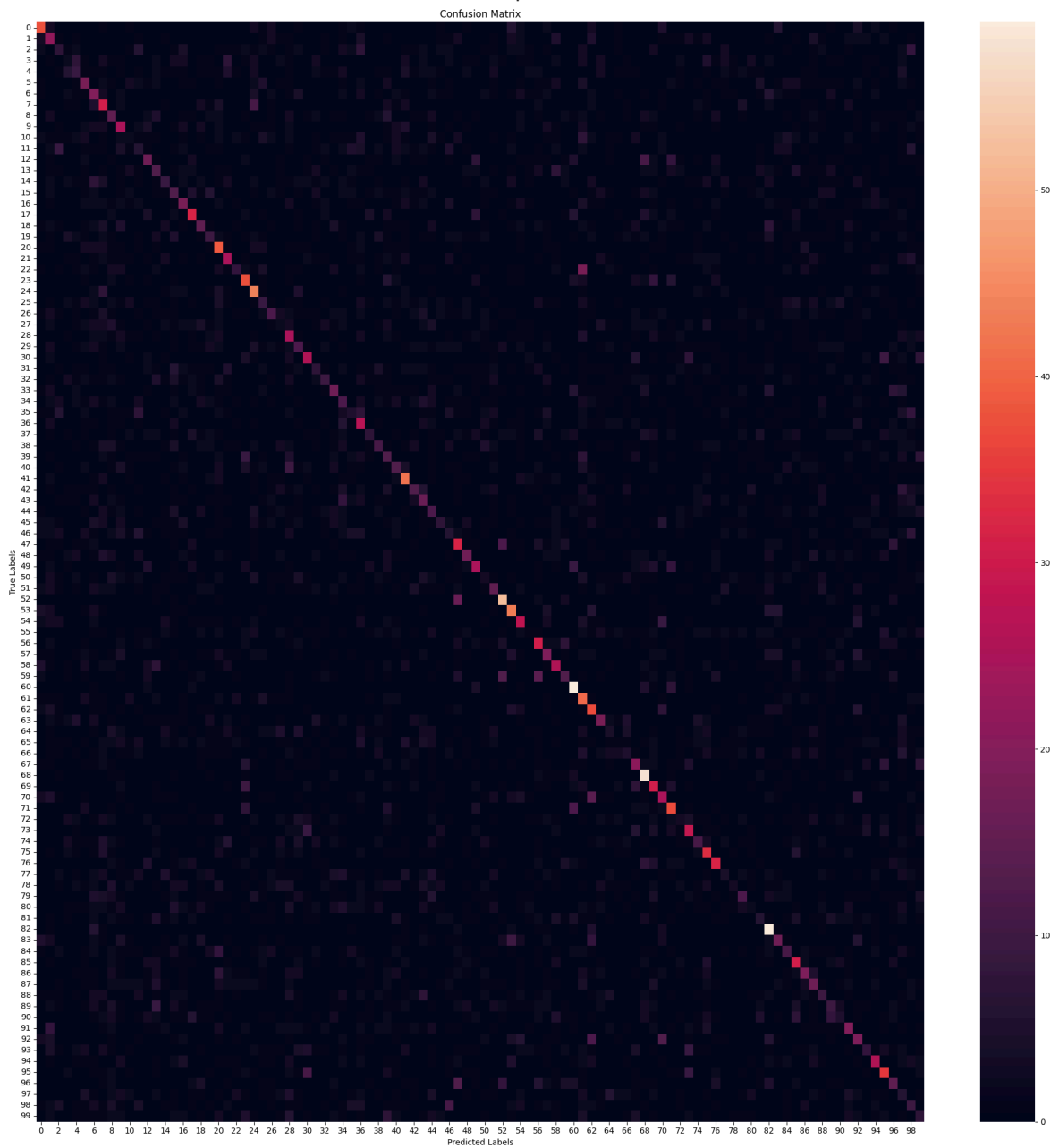
```



141/141 ————— 0s 2ms/step - accuracy: 0.2175 - loss: 3.7815

Test Accuracy: 20.91%

282/282 ————— 0s 1ms/step



	precision	recall	f1-score	support
beaver	0.41	0.42	0.42	90
dolphin	0.16	0.24	0.19	90
otter	0.11	0.08	0.09	90
seal	0.04	0.02	0.03	90
whale	0.12	0.10	0.11	90
aquarium fish	0.18	0.20	0.19	90
flatfish	0.14	0.22	0.17	90
ray	0.20	0.34	0.25	90
shark	0.12	0.17	0.14	90
trout	0.20	0.28	0.23	90
orchids	0.04	0.02	0.03	90
poppies	0.08	0.07	0.07	90
roses	0.22	0.19	0.20	90
sunflowers	0.13	0.14	0.14	90
tulips	0.15	0.10	0.12	90
bottles	0.12	0.14	0.13	90
bowls	0.19	0.20	0.19	90
cans	0.36	0.36	0.36	90
cups	0.18	0.17	0.17	90
plates	0.15	0.11	0.13	90
apples	0.24	0.43	0.31	90
mushrooms	0.27	0.28	0.27	90
oranges	0.20	0.09	0.12	90
pears	0.31	0.42	0.36	90
sweet peppers	0.32	0.49	0.39	90
clock	0.12	0.10	0.11	90
computer keyboard	0.19	0.13	0.16	90
lamp	0.03	0.02	0.03	90
telephone	0.17	0.28	0.21	90
television	0.13	0.13	0.13	90
bed	0.32	0.29	0.30	90
chair	0.11	0.08	0.09	90
couch	0.19	0.11	0.14	90
table	0.21	0.19	0.20	90
wardrobe	0.11	0.13	0.12	90
bee	0.10	0.06	0.07	90
beetle	0.18	0.30	0.23	90
butterfly	0.18	0.08	0.11	90
caterpillar	0.16	0.12	0.14	90
cockroach	0.11	0.14	0.12	90
bear	0.16	0.14	0.15	90
leopard	0.34	0.47	0.39	90
lion	0.22	0.14	0.17	90
tiger	0.15	0.18	0.16	90
wolf	0.11	0.13	0.12	90
bridge	0.17	0.08	0.11	90
castle	0.08	0.07	0.07	90
house	0.26	0.36	0.30	90
road	0.25	0.19	0.22	90
skyscraper	0.31	0.28	0.29	90
cloud	0.05	0.03	0.04	90
forest	0.18	0.17	0.17	90
mountain	0.47	0.58	0.52	90
plain	0.36	0.48	0.41	90
sea	0.26	0.31	0.28	90
camel	0.03	0.01	0.02	90
cattle	0.27	0.34	0.30	90
chimpanzee	0.21	0.21	0.21	90

elephant	0.27	0.29	0.28	90
kangaroo	0.24	0.14	0.18	90
fox	0.44	0.66	0.52	90
porcupine	0.21	0.46	0.29	90
possum	0.26	0.41	0.32	90
raccoon	0.23	0.20	0.21	90
skunk	0.08	0.04	0.06	90
crab	0.03	0.01	0.02	90
lobster	0.10	0.06	0.07	90
snail	0.23	0.23	0.23	90
spider	0.39	0.64	0.48	90
worm	0.23	0.34	0.27	90
baby	0.25	0.28	0.26	90
boy	0.34	0.41	0.37	90
girl	0.06	0.03	0.04	90
man	0.32	0.32	0.32	90
woman	0.19	0.12	0.15	90
crocodile	0.28	0.37	0.31	90
dinosaur	0.36	0.36	0.36	90
lizard	0.08	0.04	0.06	90
snake	0.13	0.03	0.05	90
turtle	0.19	0.13	0.16	90
hamster	0.06	0.02	0.03	90
mouse	0.10	0.07	0.08	90
rabbit	0.35	0.66	0.45	90
shrew	0.22	0.19	0.20	90
squirrel	0.14	0.12	0.13	90
maple	0.25	0.34	0.29	90
oak	0.26	0.21	0.23	90
palm	0.18	0.19	0.18	90
pine	0.13	0.11	0.12	90
willow	0.14	0.11	0.12	90
bicycle	0.10	0.06	0.07	90
bus	0.23	0.22	0.22	90
motorcycle	0.18	0.21	0.19	90
pickup truck	0.14	0.09	0.11	90
train	0.28	0.29	0.29	90
lawn-mower	0.32	0.39	0.35	90
rocket	0.19	0.17	0.18	90
streetcar	0.05	0.07	0.06	90
tank	0.10	0.11	0.10	90
tractor	0.08	0.08	0.08	90
accuracy			0.21	9000
macro avg	0.19	0.21	0.19	9000
weighted avg	0.19	0.21	0.19	9000

Epoch 1/20

657/657  2s 3ms/step - accuracy: 0.4455 - loss: 2.1398 - val_accuracy: 0.2088 - val_loss: 3.8216

Epoch 2/20

657/657  2s 3ms/step - accuracy: 0.4521 - loss: 2.1085 - val_accuracy: 0.2087 - val_loss: 3.8463

Epoch 3/20

657/657  2s 3ms/step - accuracy: 0.4575 - loss: 2.0825 - val_accuracy: 0.2061 - val_loss: 3.8699

Epoch 4/20

657/657  2s 3ms/step - accuracy: 0.4628 - loss: 2.0632 - val_accuracy: 0.2063 - val_loss: 3.8981

Epoch 5/20

657/657 ————— 2s 3ms/step - accuracy: 0.4644 - loss: 2.0519 - val_
accuracy: 0.2060 - val_loss: 3.9299
Epoch 6/20

657/657 ————— 2s 3ms/step - accuracy: 0.4650 - loss: 2.0488 - val_
accuracy: 0.2046 - val_loss: 3.9552
Epoch 7/20

657/657 ————— 2s 3ms/step - accuracy: 0.4642 - loss: 2.0468 - val_
accuracy: 0.2004 - val_loss: 3.9863
Epoch 8/20

657/657 ————— 2s 3ms/step - accuracy: 0.4599 - loss: 2.0560 - val_
accuracy: 0.1968 - val_loss: 4.0155
Epoch 9/20

657/657 ————— 2s 3ms/step - accuracy: 0.4583 - loss: 2.0617 - val_
accuracy: 0.1933 - val_loss: 4.0638
Epoch 10/20

657/657 ————— 2s 3ms/step - accuracy: 0.4574 - loss: 2.0572 - val_
accuracy: 0.1903 - val_loss: 4.0856
Epoch 11/20

657/657 ————— 2s 3ms/step - accuracy: 0.4584 - loss: 2.0481 - val_
accuracy: 0.1920 - val_loss: 4.0820
Epoch 12/20

657/657 ————— 2s 3ms/step - accuracy: 0.4668 - loss: 2.0282 - val_
accuracy: 0.1944 - val_loss: 4.0747
Epoch 13/20

657/657 ————— 2s 3ms/step - accuracy: 0.4716 - loss: 2.0064 - val_
accuracy: 0.1958 - val_loss: 4.0770
Epoch 14/20

657/657 ————— 2s 3ms/step - accuracy: 0.4783 - loss: 1.9875 - val_
accuracy: 0.1954 - val_loss: 4.0824
Epoch 15/20

657/657 ————— 2s 3ms/step - accuracy: 0.4817 - loss: 1.9710 - val_
accuracy: 0.1958 - val_loss: 4.0916
Epoch 16/20

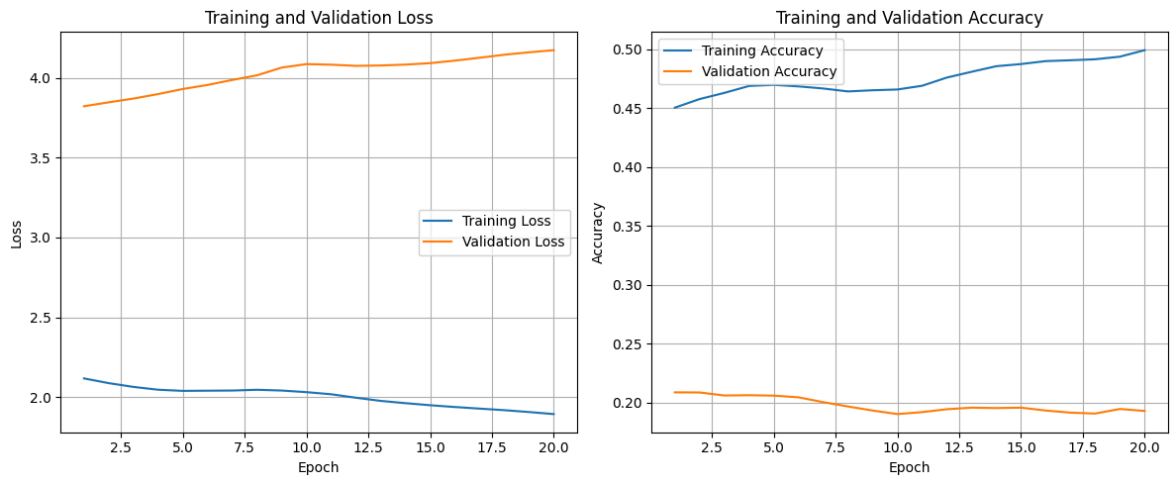
657/657 ————— 2s 3ms/step - accuracy: 0.4861 - loss: 1.9578 - val_
accuracy: 0.1933 - val_loss: 4.1074
Epoch 17/20

657/657 ————— 2s 3ms/step - accuracy: 0.4883 - loss: 1.9458 - val_
accuracy: 0.1916 - val_loss: 4.1255
Epoch 18/20

657/657 ————— 2s 3ms/step - accuracy: 0.4875 - loss: 1.9373 - val_
accuracy: 0.1908 - val_loss: 4.1443
Epoch 19/20

657/657 ————— 2s 3ms/step - accuracy: 0.4887 - loss: 1.9268 - val_
accuracy: 0.1947 - val_loss: 4.1596
Epoch 20/20

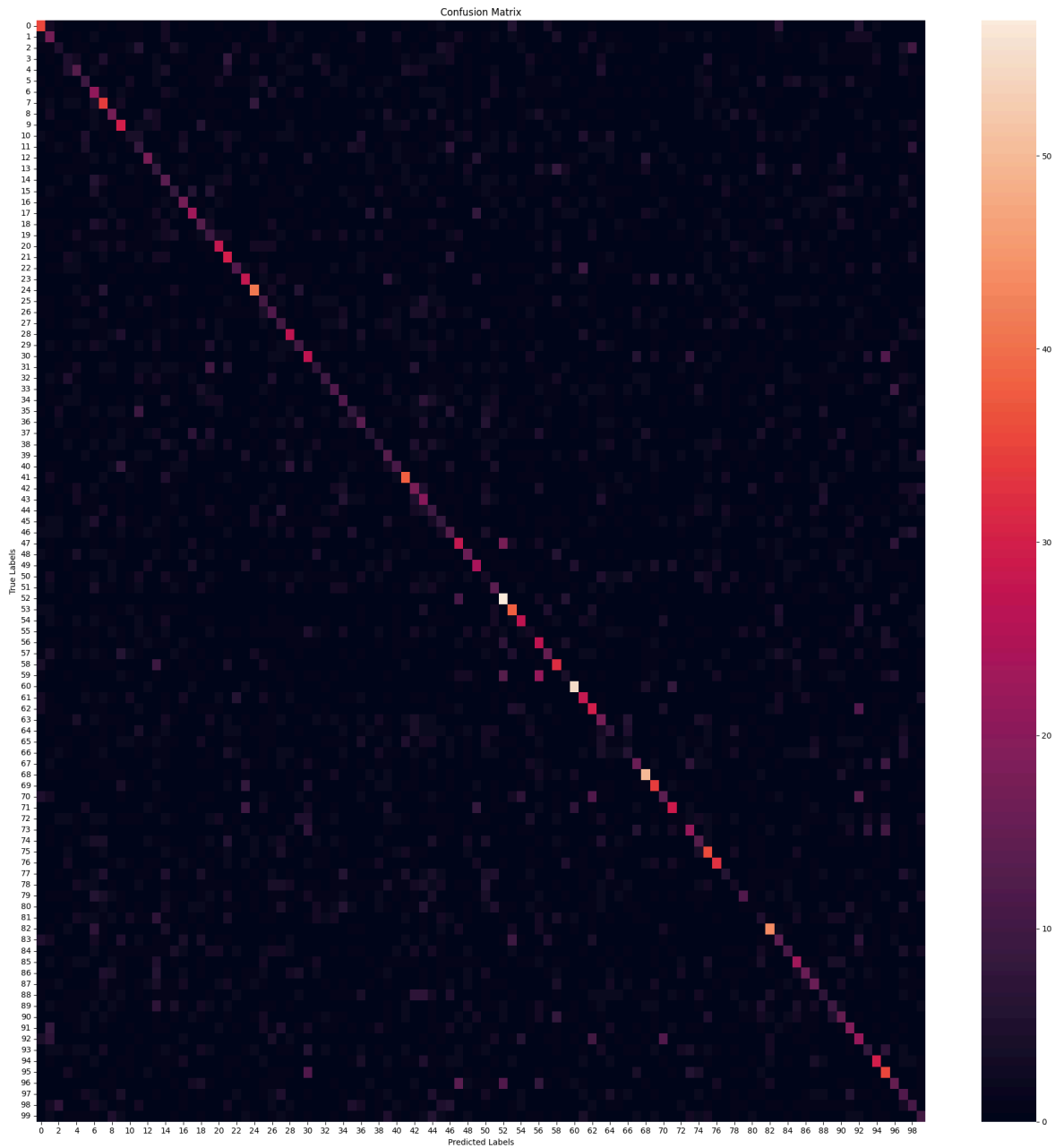
657/657 ————— 2s 3ms/step - accuracy: 0.4935 - loss: 1.9153 - val_
accuracy: 0.1930 - val_loss: 4.1727



141/141 ————— 0s 2ms/step - accuracy: 0.2036 - loss: 4.1234

Test Accuracy: 19.69%

282/282 ————— 0s 1ms/step



	precision	recall	f1-score	support
beaver	0.43	0.39	0.41	90
dolphin	0.14	0.19	0.16	90
otter	0.08	0.06	0.07	90
seal	0.07	0.06	0.06	90
whale	0.13	0.14	0.14	90
aquarium fish	0.12	0.10	0.11	90
flatfish	0.14	0.23	0.18	90
ray	0.24	0.38	0.29	90
shark	0.20	0.19	0.19	90
trout	0.24	0.33	0.28	90
orchids	0.08	0.04	0.06	90
poppies	0.08	0.09	0.09	90
roses	0.26	0.20	0.23	90
sunflowers	0.07	0.09	0.08	90
tulips	0.11	0.16	0.13	90
bottles	0.10	0.09	0.10	90
bowls	0.20	0.19	0.20	90
cans	0.34	0.26	0.29	90
cups	0.14	0.14	0.14	90
plates	0.08	0.10	0.09	90
apples	0.35	0.31	0.33	90
mushrooms	0.22	0.33	0.26	90
oranges	0.26	0.13	0.18	90
pears	0.34	0.31	0.33	90
sweet peppers	0.39	0.46	0.42	90
clock	0.11	0.11	0.11	90
computer keyboard	0.11	0.13	0.12	90
lamp	0.10	0.11	0.11	90
telephone	0.25	0.30	0.27	90
television	0.14	0.11	0.12	90
bed	0.24	0.30	0.27	90
chair	0.10	0.08	0.09	90
couch	0.15	0.10	0.12	90
table	0.18	0.13	0.15	90
wardrobe	0.12	0.13	0.13	90
bee	0.14	0.09	0.11	90
beetle	0.14	0.16	0.15	90
butterfly	0.16	0.07	0.09	90
caterpillar	0.11	0.08	0.09	90
cockroach	0.12	0.14	0.13	90
bear	0.16	0.12	0.14	90
leopard	0.36	0.42	0.39	90
lion	0.15	0.20	0.17	90
tiger	0.15	0.22	0.18	90
wolf	0.07	0.10	0.08	90
bridge	0.08	0.09	0.09	90
castle	0.12	0.14	0.13	90
house	0.29	0.31	0.30	90
road	0.18	0.18	0.18	90
skyscraper	0.33	0.27	0.29	90
cloud	0.03	0.03	0.03	90
forest	0.13	0.16	0.14	90
mountain	0.42	0.63	0.50	90
plain	0.35	0.42	0.38	90
sea	0.31	0.30	0.31	90
camel	0.05	0.03	0.04	90
cattle	0.18	0.30	0.23	90
chimpanzee	0.25	0.17	0.20	90

elephant	0.24	0.36	0.29	90
kangaroo	0.09	0.04	0.06	90
fox	0.57	0.61	0.59	90
porcupine	0.27	0.31	0.29	90
possum	0.27	0.33	0.30	90
raccoon	0.15	0.19	0.17	90
skunk	0.09	0.08	0.08	90
crab	0.03	0.01	0.02	90
lobster	0.11	0.07	0.08	90
snail	0.23	0.18	0.20	90
spider	0.52	0.56	0.53	90
worm	0.32	0.38	0.35	90
baby	0.23	0.16	0.19	90
boy	0.38	0.32	0.35	90
girl	0.02	0.01	0.01	90
man	0.30	0.24	0.27	90
woman	0.14	0.14	0.14	90
crocodile	0.27	0.40	0.33	90
dinosaur	0.45	0.37	0.40	90
lizard	0.10	0.07	0.08	90
snake	0.05	0.03	0.04	90
turtle	0.23	0.13	0.17	90
hamster	0.02	0.01	0.01	90
mouse	0.05	0.06	0.05	90
rabbit	0.39	0.49	0.43	90
shrew	0.17	0.16	0.16	90
squirrel	0.14	0.12	0.13	90
maple	0.24	0.26	0.25	90
oak	0.27	0.18	0.21	90
palm	0.18	0.18	0.18	90
pine	0.09	0.08	0.08	90
willow	0.15	0.10	0.12	90
bicycle	0.16	0.17	0.16	90
bus	0.18	0.21	0.19	90
motorcycle	0.18	0.24	0.21	90
pickup truck	0.10	0.09	0.10	90
train	0.33	0.33	0.33	90
lawn-mower	0.28	0.39	0.33	90
rocket	0.22	0.17	0.19	90
streetcar	0.10	0.13	0.11	90
tank	0.11	0.12	0.12	90
tractor	0.12	0.11	0.12	90
accuracy			0.20	9000
macro avg	0.19	0.20	0.19	9000
weighted avg	0.19	0.20	0.19	9000

Epoch 1/20

657/657  2s 3ms/step - accuracy: 0.4956 - loss: 1.9020 - val_accuracy: 0.1936 - val_loss: 4.1809

Epoch 2/20

657/657  2s 3ms/step - accuracy: 0.4983 - loss: 1.8896 - val_accuracy: 0.1972 - val_loss: 4.1890

Epoch 3/20

657/657  2s 3ms/step - accuracy: 0.5014 - loss: 1.8770 - val_accuracy: 0.1972 - val_loss: 4.2060

Epoch 4/20

657/657  2s 3ms/step - accuracy: 0.5037 - loss: 1.8628 - val_accuracy: 0.1943 - val_loss: 4.2312

Epoch 5/20

657/657 ————— 2s 3ms/step - accuracy: 0.5043 - loss: 1.8526 - val_
accuracy: 0.1939 - val_loss: 4.2670
Epoch 6/20

657/657 ————— 2s 3ms/step - accuracy: 0.5036 - loss: 1.8522 - val_
accuracy: 0.1904 - val_loss: 4.3014
Epoch 7/20

657/657 ————— 2s 3ms/step - accuracy: 0.5079 - loss: 1.8500 - val_
accuracy: 0.1916 - val_loss: 4.3275
Epoch 8/20

657/657 ————— 2s 3ms/step - accuracy: 0.5067 - loss: 1.8413 - val_
accuracy: 0.1959 - val_loss: 4.3594
Epoch 9/20

657/657 ————— 2s 3ms/step - accuracy: 0.5116 - loss: 1.8273 - val_
accuracy: 0.1952 - val_loss: 4.3761
Epoch 10/20

657/657 ————— 2s 3ms/step - accuracy: 0.5165 - loss: 1.8070 - val_
accuracy: 0.1950 - val_loss: 4.3854
Epoch 11/20

657/657 ————— 2s 3ms/step - accuracy: 0.5198 - loss: 1.7859 - val_
accuracy: 0.1959 - val_loss: 4.3984
Epoch 12/20

657/657 ————— 2s 3ms/step - accuracy: 0.5258 - loss: 1.7679 - val_
accuracy: 0.1997 - val_loss: 4.4112
Epoch 13/20

657/657 ————— 2s 3ms/step - accuracy: 0.5262 - loss: 1.7581 - val_
accuracy: 0.1978 - val_loss: 4.4233
Epoch 14/20

657/657 ————— 2s 3ms/step - accuracy: 0.5254 - loss: 1.7606 - val_
accuracy: 0.1967 - val_loss: 4.4385
Epoch 15/20

657/657 ————— 2s 3ms/step - accuracy: 0.5187 - loss: 1.7804 - val_
accuracy: 0.1949 - val_loss: 4.4510
Epoch 16/20

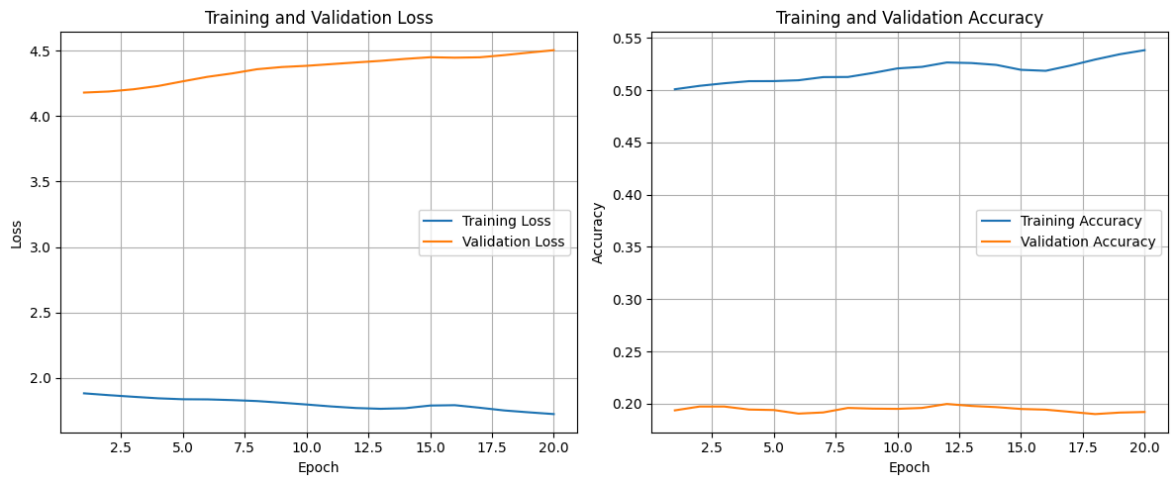
657/657 ————— 2s 3ms/step - accuracy: 0.5195 - loss: 1.7882 - val_
accuracy: 0.1942 - val_loss: 4.4476
Epoch 17/20

657/657 ————— 2s 3ms/step - accuracy: 0.5243 - loss: 1.7652 - val_
accuracy: 0.1921 - val_loss: 4.4504
Epoch 18/20

657/657 ————— 2s 3ms/step - accuracy: 0.5322 - loss: 1.7385 - val_
accuracy: 0.1900 - val_loss: 4.4672
Epoch 19/20

657/657 ————— 2s 3ms/step - accuracy: 0.5388 - loss: 1.7180 - val_
accuracy: 0.1914 - val_loss: 4.4864
Epoch 20/20

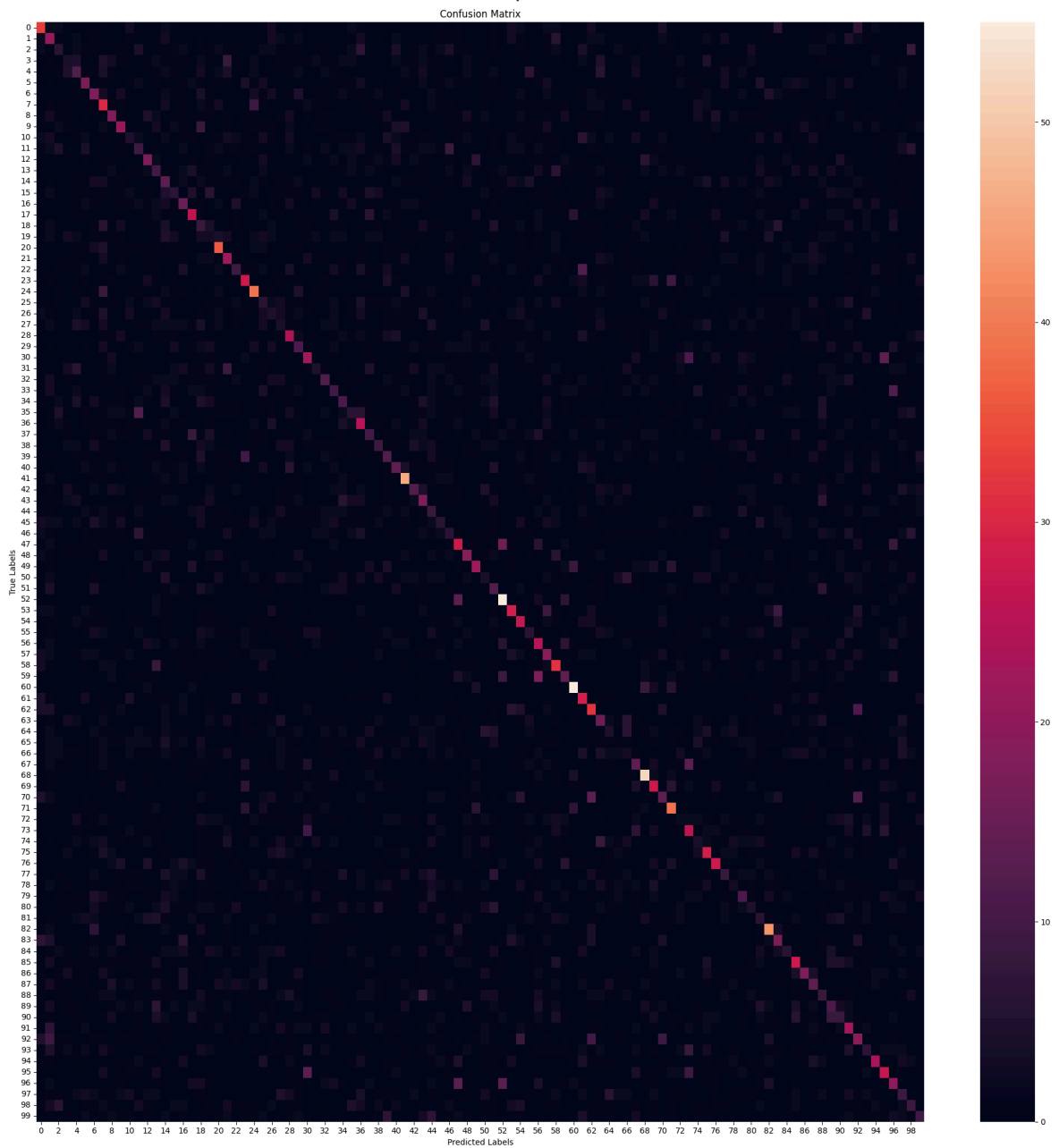
657/657 ————— 2s 3ms/step - accuracy: 0.5444 - loss: 1.7040 - val_
accuracy: 0.1920 - val_loss: 4.5048



141/141 ————— 0s 2ms/step - accuracy: 0.1896 - loss: 4.5084

Test Accuracy: 19.00%

282/282 ————— 0s 1ms/step



	precision	recall	f1-score	support
beaver	0.33	0.37	0.35	90
dolphin	0.14	0.23	0.17	90
otter	0.10	0.07	0.08	90
seal	0.06	0.04	0.05	90
whale	0.14	0.13	0.14	90
aquarium fish	0.20	0.19	0.20	90
flatfish	0.17	0.20	0.19	90
ray	0.20	0.33	0.25	90
shark	0.21	0.20	0.21	90
trout	0.24	0.23	0.23	90
orchids	0.08	0.06	0.07	90
poppies	0.11	0.10	0.10	90
roses	0.22	0.20	0.21	90
sunflowers	0.09	0.11	0.10	90
tulips	0.09	0.14	0.11	90
bottles	0.10	0.08	0.09	90
bowls	0.17	0.17	0.17	90
cans	0.30	0.29	0.30	90
cups	0.06	0.09	0.07	90
plates	0.03	0.03	0.03	90
apples	0.37	0.40	0.38	90
mushrooms	0.21	0.24	0.22	90
oranges	0.17	0.10	0.13	90
pears	0.32	0.31	0.32	90
sweet peppers	0.34	0.43	0.38	90
clock	0.06	0.07	0.06	90
computer keyboard	0.06	0.06	0.06	90
lamp	0.05	0.04	0.05	90
telephone	0.22	0.27	0.24	90
television	0.12	0.12	0.12	90
bed	0.23	0.24	0.24	90
chair	0.10	0.06	0.07	90
couch	0.16	0.13	0.15	90
table	0.13	0.11	0.12	90
wardrobe	0.14	0.12	0.13	90
bee	0.10	0.07	0.08	90
beetle	0.21	0.28	0.24	90
butterfly	0.14	0.11	0.13	90
caterpillar	0.13	0.10	0.11	90
cockroach	0.14	0.13	0.14	90
bear	0.15	0.16	0.15	90
leopard	0.40	0.51	0.45	90
lion	0.20	0.13	0.16	90
tiger	0.16	0.20	0.18	90
wolf	0.07	0.09	0.08	90
bridge	0.10	0.07	0.08	90
castle	0.08	0.07	0.07	90
house	0.24	0.31	0.27	90
road	0.21	0.21	0.21	90
skyscraper	0.33	0.24	0.28	90
cloud	0.05	0.04	0.05	90
forest	0.10	0.13	0.12	90
mountain	0.46	0.61	0.53	90
plain	0.33	0.32	0.33	90
sea	0.24	0.30	0.27	90
camel	0.10	0.07	0.08	90
cattle	0.19	0.28	0.23	90
chimpanzee	0.20	0.21	0.21	90

elephant	0.26	0.36	0.30	90
kangaroo	0.20	0.16	0.17	90
fox	0.53	0.61	0.57	90
porcupine	0.24	0.32	0.27	90
possum	0.26	0.36	0.30	90
raccoon	0.14	0.17	0.15	90
skunk	0.08	0.06	0.07	90
crab	0.03	0.01	0.02	90
lobster	0.04	0.03	0.04	90
snail	0.17	0.16	0.16	90
spider	0.46	0.58	0.51	90
worm	0.25	0.31	0.27	90
baby	0.18	0.16	0.17	90
boy	0.37	0.43	0.40	90
girl	0.04	0.02	0.03	90
man	0.27	0.29	0.28	90
woman	0.06	0.04	0.05	90
crocodile	0.26	0.32	0.29	90
dinosaur	0.36	0.31	0.34	90
lizard	0.12	0.09	0.10	90
snake	0.06	0.03	0.04	90
turtle	0.16	0.12	0.14	90
hamster	0.07	0.06	0.06	90
mouse	0.06	0.07	0.06	90
rabbit	0.37	0.48	0.42	90
shrew	0.17	0.19	0.18	90
squirrel	0.12	0.07	0.09	90
maple	0.26	0.31	0.29	90
oak	0.30	0.20	0.24	90
palm	0.15	0.16	0.15	90
pine	0.09	0.09	0.09	90
willow	0.12	0.12	0.12	90
bicycle	0.11	0.09	0.10	90
bus	0.16	0.24	0.19	90
motorcycle	0.18	0.21	0.20	90
pickup truck	0.10	0.08	0.09	90
train	0.32	0.26	0.28	90
lawn-mower	0.32	0.30	0.31	90
rocket	0.17	0.22	0.19	90
streetcar	0.09	0.10	0.09	90
tank	0.11	0.10	0.10	90
tractor	0.16	0.11	0.13	90
accuracy			0.19	9000
macro avg	0.18	0.19	0.18	9000
weighted avg	0.18	0.19	0.18	9000

Epoch 1/20

657/657  2s 3ms/step - accuracy: 0.5444 - loss: 1.6962 - val_accuracy: 0.1901 - val_loss: 4.5276

Epoch 2/20

657/657  2s 3ms/step - accuracy: 0.5443 - loss: 1.6908 - val_accuracy: 0.1886 - val_loss: 4.5456

Epoch 3/20

657/657  2s 3ms/step - accuracy: 0.5428 - loss: 1.6847 - val_accuracy: 0.1884 - val_loss: 4.5734

Epoch 4/20

657/657  2s 3ms/step - accuracy: 0.5403 - loss: 1.6863 - val_accuracy: 0.1846 - val_loss: 4.6134

Epoch 5/20

657/657 ————— 2s 3ms/step - accuracy: 0.5378 - loss: 1.6990 - val_
accuracy: 0.1888 - val_loss: 4.6231
Epoch 6/20

657/657 ————— 2s 3ms/step - accuracy: 0.5336 - loss: 1.7042 - val_
accuracy: 0.1881 - val_loss: 4.6193
Epoch 7/20

657/657 ————— 2s 3ms/step - accuracy: 0.5352 - loss: 1.6994 - val_
accuracy: 0.1840 - val_loss: 4.6095
Epoch 8/20

657/657 ————— 2s 3ms/step - accuracy: 0.5399 - loss: 1.6841 - val_
accuracy: 0.1840 - val_loss: 4.6054
Epoch 9/20

657/657 ————— 2s 3ms/step - accuracy: 0.5439 - loss: 1.6622 - val_
accuracy: 0.1846 - val_loss: 4.6255
Epoch 10/20

657/657 ————— 2s 3ms/step - accuracy: 0.5531 - loss: 1.6362 - val_
accuracy: 0.1850 - val_loss: 4.6444
Epoch 11/20

657/657 ————— 2s 3ms/step - accuracy: 0.5622 - loss: 1.6145 - val_
accuracy: 0.1837 - val_loss: 4.6687
Epoch 12/20

657/657 ————— 2s 3ms/step - accuracy: 0.5677 - loss: 1.5959 - val_
accuracy: 0.1841 - val_loss: 4.6975
Epoch 13/20

657/657 ————— 2s 3ms/step - accuracy: 0.5722 - loss: 1.5764 - val_
accuracy: 0.1831 - val_loss: 4.7288
Epoch 14/20

657/657 ————— 2s 3ms/step - accuracy: 0.5784 - loss: 1.5590 - val_
accuracy: 0.1824 - val_loss: 4.7649
Epoch 15/20

657/657 ————— 2s 3ms/step - accuracy: 0.5832 - loss: 1.5455 - val_
accuracy: 0.1830 - val_loss: 4.7888
Epoch 16/20

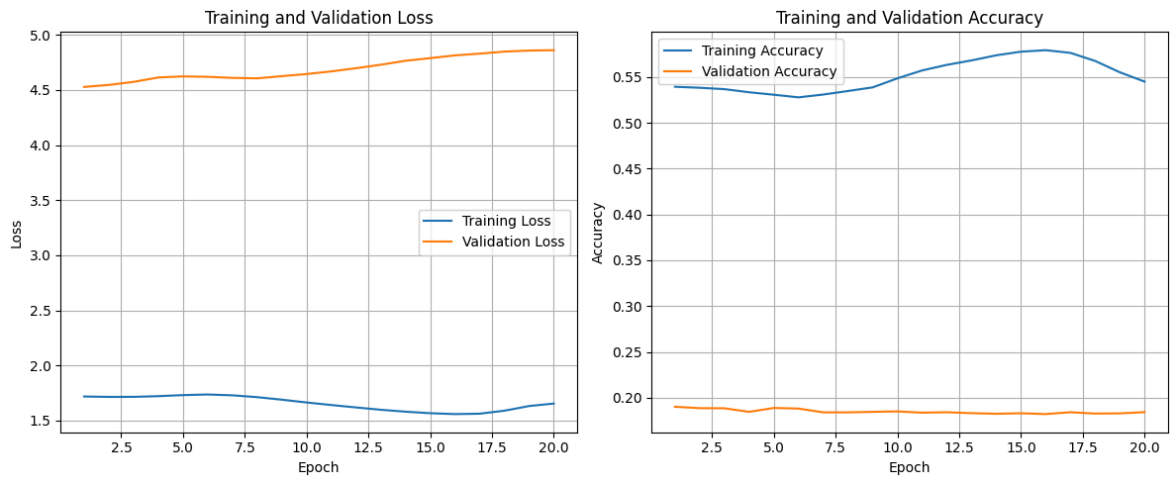
657/657 ————— 2s 3ms/step - accuracy: 0.5833 - loss: 1.5362 - val_
accuracy: 0.1821 - val_loss: 4.8138
Epoch 17/20

657/657 ————— 2s 3ms/step - accuracy: 0.5835 - loss: 1.5371 - val_
accuracy: 0.1841 - val_loss: 4.8297
Epoch 18/20

657/657 ————— 2s 3ms/step - accuracy: 0.5743 - loss: 1.5597 - val_
accuracy: 0.1827 - val_loss: 4.8481
Epoch 19/20

657/657 ————— 2s 3ms/step - accuracy: 0.5630 - loss: 1.5949 - val_
accuracy: 0.1829 - val_loss: 4.8576
Epoch 20/20

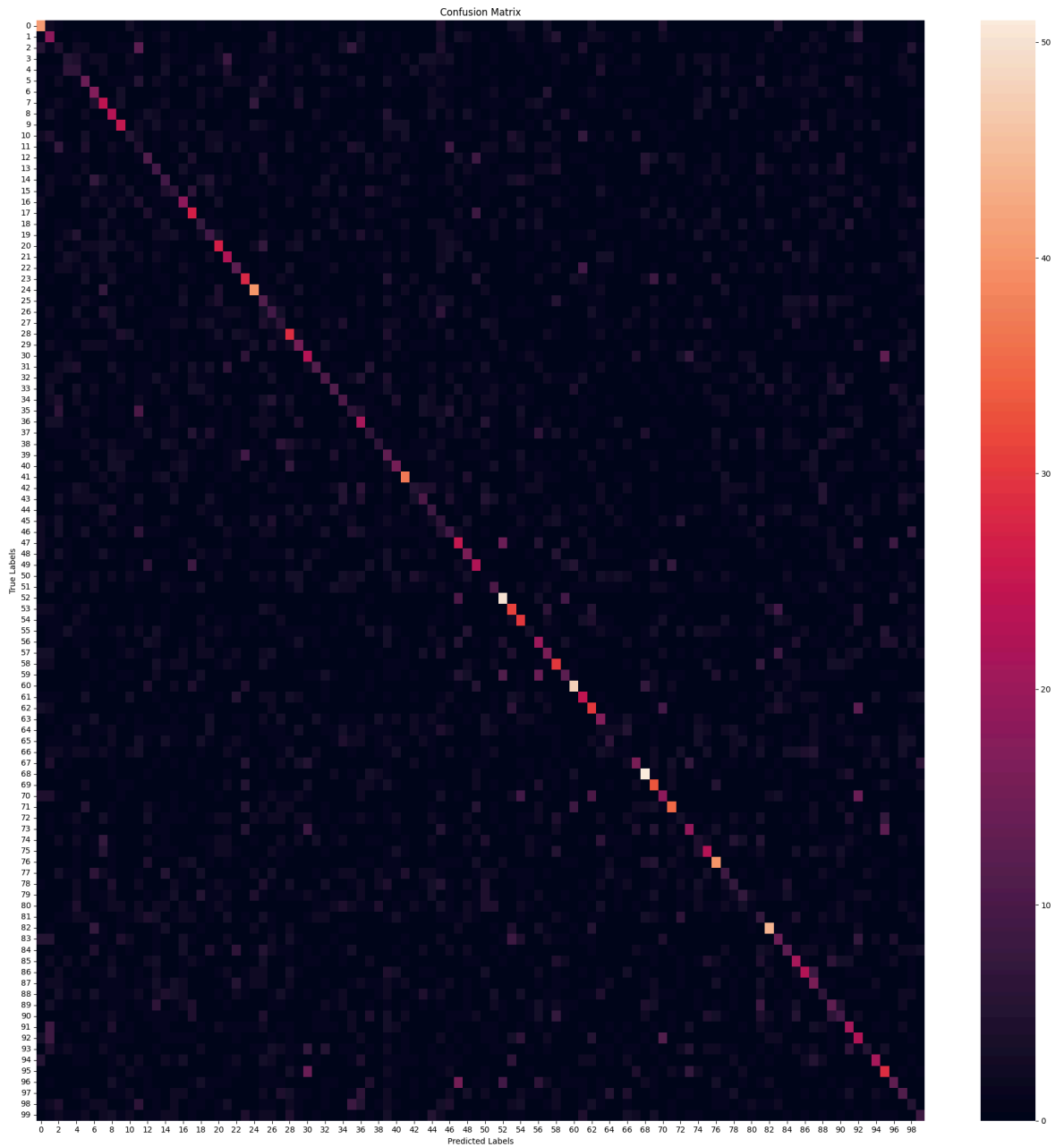
657/657 ————— 2s 3ms/step - accuracy: 0.5509 - loss: 1.6210 - val_
accuracy: 0.1842 - val_loss: 4.8609



141/141 ————— 0s 2ms/step - accuracy: 0.1810 - loss: 4.8245

Test Accuracy: 18.23%

282/282 ————— 0s 1ms/step



	precision	recall	f1-score	support
beaver	0.39	0.46	0.42	90
dolphin	0.14	0.20	0.17	90
otter	0.04	0.04	0.04	90
seal	0.08	0.06	0.07	90
whale	0.06	0.07	0.07	90
aquarium fish	0.14	0.16	0.15	90
flatfish	0.17	0.19	0.18	90
ray	0.22	0.27	0.24	90
shark	0.17	0.26	0.21	90
trout	0.27	0.28	0.27	90
orchids	0.08	0.06	0.07	90
poppies	0.04	0.04	0.04	90
roses	0.15	0.12	0.13	90
sunflowers	0.10	0.10	0.10	90
tulips	0.09	0.10	0.10	90
bottles	0.08	0.07	0.07	90
bowls	0.19	0.20	0.19	90
cans	0.27	0.30	0.28	90
cups	0.09	0.08	0.08	90
plates	0.10	0.10	0.10	90
apples	0.26	0.30	0.28	90
mushrooms	0.21	0.24	0.23	90
oranges	0.17	0.13	0.15	90
pears	0.29	0.31	0.30	90
sweet peppers	0.38	0.44	0.41	90
clock	0.08	0.12	0.10	90
computer keyboard	0.08	0.10	0.09	90
lamp	0.09	0.07	0.08	90
telephone	0.22	0.32	0.26	90
television	0.17	0.17	0.17	90
bed	0.26	0.26	0.26	90
chair	0.18	0.12	0.15	90
couch	0.12	0.11	0.12	90
table	0.18	0.12	0.15	90
wardrobe	0.12	0.11	0.11	90
bee	0.08	0.07	0.07	90
beetle	0.18	0.23	0.20	90
butterfly	0.09	0.07	0.08	90
caterpillar	0.12	0.07	0.09	90
cockroach	0.09	0.14	0.11	90
bear	0.17	0.17	0.17	90
leopard	0.39	0.41	0.40	90
lion	0.11	0.06	0.07	90
tiger	0.16	0.11	0.13	90
wolf	0.09	0.09	0.09	90
bridge	0.05	0.07	0.06	90
castle	0.09	0.10	0.09	90
house	0.24	0.28	0.26	90
road	0.16	0.18	0.17	90
skyscraper	0.26	0.24	0.25	90
cloud	0.01	0.01	0.01	90
forest	0.12	0.11	0.11	90
mountain	0.47	0.56	0.51	90
plain	0.31	0.34	0.33	90
sea	0.23	0.33	0.27	90
camel	0.07	0.03	0.05	90
cattle	0.17	0.22	0.19	90
chimpanzee	0.18	0.18	0.18	90

elephant	0.24	0.33	0.28	90
kangaroo	0.21	0.13	0.16	90
fox	0.52	0.53	0.52	90
porcupine	0.26	0.27	0.26	90
possum	0.28	0.33	0.31	90
raccoon	0.17	0.19	0.18	90
skunk	0.04	0.02	0.03	90
crab	0.00	0.00	0.00	90
lobster	0.02	0.01	0.01	90
snail	0.27	0.18	0.21	90
spider	0.46	0.57	0.50	90
worm	0.24	0.37	0.29	90
baby	0.22	0.20	0.21	90
boy	0.39	0.39	0.39	90
girl	0.05	0.03	0.04	90
man	0.30	0.21	0.25	90
woman	0.06	0.04	0.05	90
crocodile	0.25	0.26	0.25	90
dinosaur	0.35	0.44	0.39	90
lizard	0.12	0.09	0.10	90
snake	0.10	0.08	0.09	90
turtle	0.12	0.08	0.10	90
hamster	0.03	0.01	0.02	90
mouse	0.06	0.08	0.06	90
rabbit	0.46	0.49	0.47	90
shrew	0.16	0.16	0.16	90
squirrel	0.13	0.13	0.13	90
maple	0.22	0.23	0.23	90
oak	0.25	0.26	0.25	90
palm	0.13	0.18	0.15	90
pine	0.08	0.07	0.07	90
willow	0.13	0.14	0.14	90
bicycle	0.10	0.10	0.10	90
bus	0.19	0.23	0.21	90
motorcycle	0.15	0.24	0.19	90
pickup truck	0.13	0.06	0.08	90
train	0.29	0.23	0.26	90
lawn-mower	0.27	0.32	0.29	90
rocket	0.24	0.14	0.18	90
streetcar	0.11	0.12	0.12	90
tank	0.05	0.06	0.05	90
tractor	0.13	0.09	0.11	90
accuracy			0.18	9000
macro avg	0.17	0.18	0.18	9000
weighted avg	0.17	0.18	0.18	9000

Epoch 1/20

657/657 ————— 2s 3ms/step - accuracy: 0.5526 - loss: 1.6170 - val_
accuracy: 0.1861 - val_loss: 4.8532

Epoch 2/20

657/657 ————— 2s 3ms/step - accuracy: 0.5595 - loss: 1.5991 - val_
accuracy: 0.1849 - val_loss: 4.8603

Epoch 3/20

657/657 ————— 2s 3ms/step - accuracy: 0.5635 - loss: 1.5824 - val_
accuracy: 0.1853 - val_loss: 4.8752

Epoch 4/20

657/657 ————— 2s 3ms/step - accuracy: 0.5674 - loss: 1.5622 - val_
accuracy: 0.1857 - val_loss: 4.8924

Epoch 5/20

657/657 ————— 2s 3ms/step - accuracy: 0.5733 - loss: 1.5388 - val_
accuracy: 0.1858 - val_loss: 4.9132
Epoch 6/20

657/657 ————— 2s 3ms/step - accuracy: 0.5807 - loss: 1.5228 - val_
accuracy: 0.1857 - val_loss: 4.9308
Epoch 7/20

657/657 ————— 2s 3ms/step - accuracy: 0.5885 - loss: 1.5076 - val_
accuracy: 0.1843 - val_loss: 4.9591
Epoch 8/20

657/657 ————— 2s 3ms/step - accuracy: 0.5928 - loss: 1.4976 - val_
accuracy: 0.1837 - val_loss: 4.9811
Epoch 9/20

657/657 ————— 2s 3ms/step - accuracy: 0.5934 - loss: 1.4946 - val_
accuracy: 0.1830 - val_loss: 5.0031
Epoch 10/20

657/657 ————— 2s 3ms/step - accuracy: 0.5866 - loss: 1.5077 - val_
accuracy: 0.1851 - val_loss: 5.0139
Epoch 11/20

657/657 ————— 2s 3ms/step - accuracy: 0.5742 - loss: 1.5413 - val_
accuracy: 0.1836 - val_loss: 5.0070
Epoch 12/20

657/657 ————— 2s 3ms/step - accuracy: 0.5679 - loss: 1.5570 - val_
accuracy: 0.1861 - val_loss: 5.0239
Epoch 13/20

657/657 ————— 2s 3ms/step - accuracy: 0.5706 - loss: 1.5490 - val_
accuracy: 0.1802 - val_loss: 5.0239
Epoch 14/20

657/657 ————— 2s 3ms/step - accuracy: 0.5750 - loss: 1.5310 - val_
accuracy: 0.1807 - val_loss: 5.0377
Epoch 15/20

657/657 ————— 2s 3ms/step - accuracy: 0.5777 - loss: 1.5171 - val_
accuracy: 0.1796 - val_loss: 5.0536
Epoch 16/20

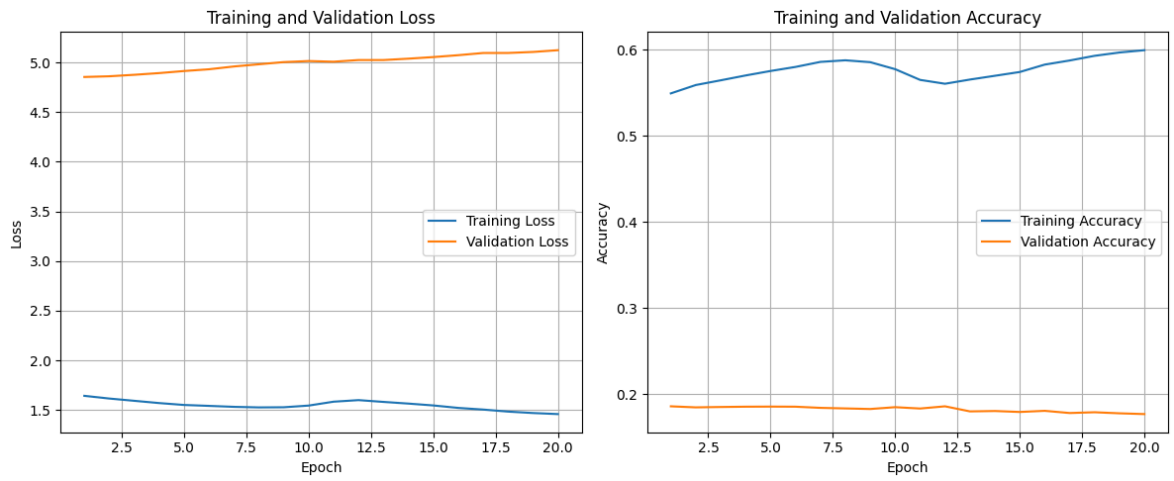
657/657 ————— 2s 3ms/step - accuracy: 0.5858 - loss: 1.4917 - val_
accuracy: 0.1809 - val_loss: 5.0727
Epoch 17/20

657/657 ————— 2s 3ms/step - accuracy: 0.5918 - loss: 1.4783 - val_
accuracy: 0.1783 - val_loss: 5.0948
Epoch 18/20

657/657 ————— 2s 3ms/step - accuracy: 0.5990 - loss: 1.4585 - val_
accuracy: 0.1792 - val_loss: 5.0951
Epoch 19/20

657/657 ————— 2s 3ms/step - accuracy: 0.6004 - loss: 1.4431 - val_
accuracy: 0.1780 - val_loss: 5.1053
Epoch 20/20

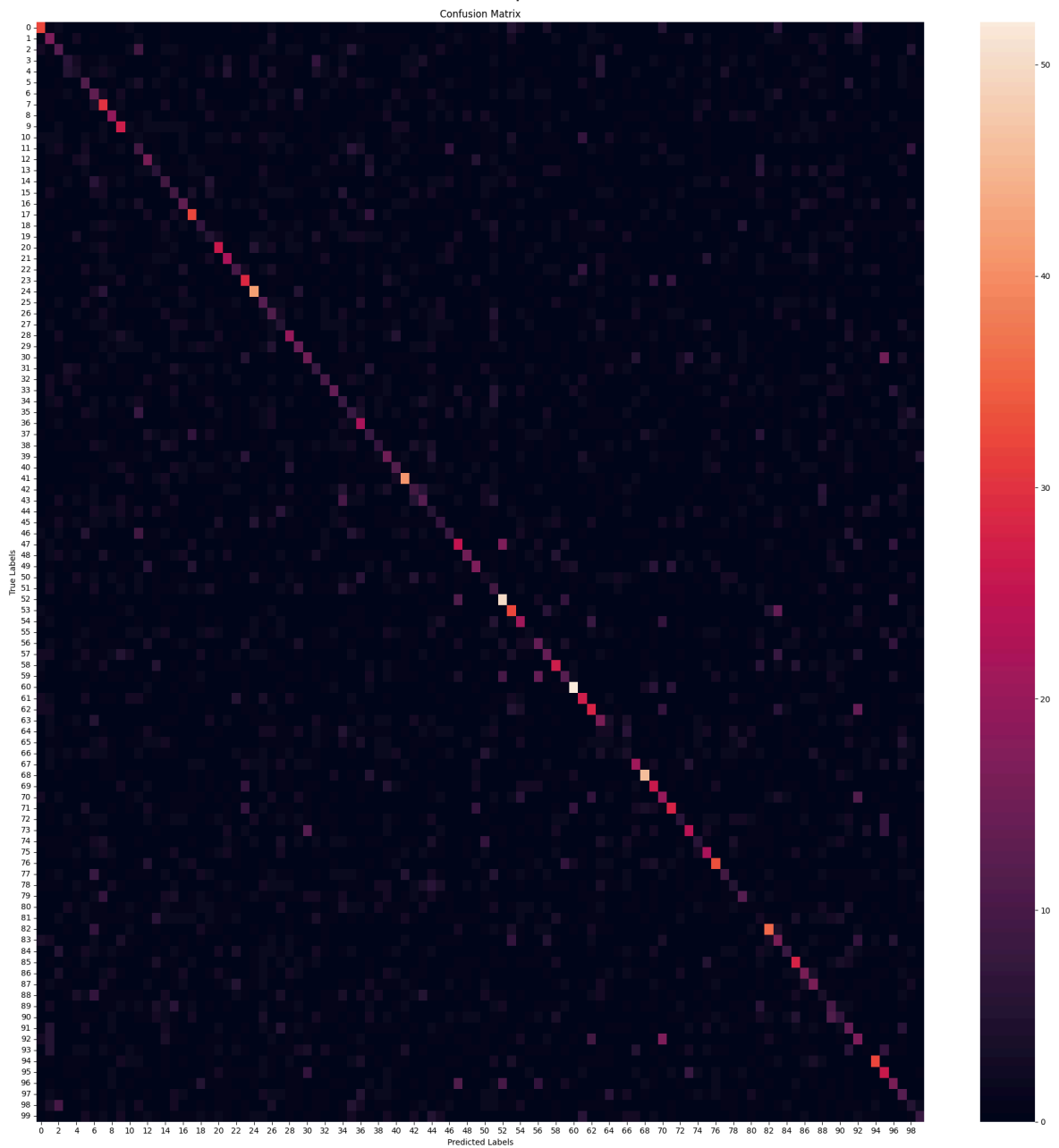
657/657 ————— 2s 3ms/step - accuracy: 0.6037 - loss: 1.4358 - val_
accuracy: 0.1771 - val_loss: 5.1230



141/141 ————— 0s 2ms/step - accuracy: 0.1769 - loss: 5.0844

Test Accuracy: 17.59%

282/282 ————— 0s 1ms/step



	precision	recall	f1-score	support
beaver	0.39	0.36	0.37	90
dolphin	0.14	0.19	0.16	90
otter	0.13	0.13	0.13	90
seal	0.11	0.07	0.08	90
whale	0.05	0.04	0.05	90
aquarium fish	0.11	0.12	0.11	90
flatfish	0.10	0.14	0.12	90
ray	0.21	0.33	0.26	90
shark	0.20	0.21	0.21	90
trout	0.25	0.30	0.27	90
orchids	0.02	0.01	0.01	90
poppies	0.08	0.10	0.09	90
roses	0.18	0.18	0.18	90
sunflowers	0.08	0.07	0.07	90
tulips	0.09	0.10	0.10	90
bottles	0.08	0.10	0.09	90
bowls	0.15	0.14	0.15	90
cans	0.32	0.36	0.34	90
cups	0.10	0.08	0.09	90
plates	0.05	0.06	0.05	90
apples	0.25	0.29	0.27	90
mushrooms	0.26	0.24	0.25	90
oranges	0.13	0.11	0.12	90
pears	0.34	0.32	0.33	90
sweet peppers	0.36	0.47	0.41	90
clock	0.11	0.13	0.12	90
computer keyboard	0.09	0.12	0.10	90
lamp	0.06	0.06	0.06	90
telephone	0.19	0.22	0.21	90
television	0.15	0.16	0.15	90
bed	0.24	0.17	0.20	90
chair	0.09	0.09	0.09	90
couch	0.14	0.11	0.12	90
table	0.19	0.16	0.17	90
wardrobe	0.07	0.09	0.08	90
bee	0.08	0.09	0.09	90
beetle	0.17	0.24	0.20	90
butterfly	0.10	0.09	0.09	90
caterpillar	0.12	0.08	0.09	90
cockroach	0.13	0.17	0.14	90
bear	0.13	0.12	0.12	90
leopard	0.35	0.46	0.39	90
lion	0.11	0.10	0.11	90
tiger	0.15	0.13	0.14	90
wolf	0.04	0.06	0.05	90
bridge	0.09	0.08	0.08	90
castle	0.12	0.09	0.10	90
house	0.26	0.28	0.27	90
road	0.24	0.17	0.20	90
skyscraper	0.28	0.19	0.23	90
cloud	0.03	0.03	0.03	90
forest	0.06	0.10	0.08	90
mountain	0.43	0.56	0.48	90
plain	0.30	0.36	0.33	90
sea	0.19	0.23	0.21	90
camel	0.04	0.02	0.03	90
cattle	0.17	0.16	0.16	90
chimpanzee	0.17	0.16	0.16	90

elephant	0.28	0.30	0.29	90
kangaroo	0.18	0.13	0.15	90
fox	0.53	0.58	0.55	90
porcupine	0.27	0.30	0.29	90
possum	0.26	0.31	0.28	90
raccoon	0.15	0.18	0.16	90
skunk	0.04	0.03	0.04	90
crab	0.03	0.01	0.02	90
lobster	0.06	0.04	0.05	90
snail	0.28	0.23	0.25	90
spider	0.51	0.51	0.51	90
worm	0.24	0.29	0.26	90
baby	0.22	0.22	0.22	90
boy	0.35	0.31	0.33	90
girl	0.08	0.07	0.07	90
man	0.38	0.27	0.31	90
woman	0.11	0.07	0.08	90
crocodile	0.21	0.24	0.23	90
dinosaur	0.37	0.37	0.37	90
lizard	0.13	0.10	0.11	90
snake	0.07	0.06	0.06	90
turtle	0.18	0.13	0.15	90
hamster	0.02	0.01	0.02	90
mouse	0.03	0.03	0.03	90
rabbit	0.36	0.40	0.38	90
shrew	0.15	0.18	0.16	90
squirrel	0.12	0.09	0.10	90
maple	0.25	0.31	0.28	90
oak	0.21	0.18	0.19	90
palm	0.17	0.18	0.17	90
pine	0.05	0.04	0.04	90
willow	0.10	0.11	0.10	90
bicycle	0.09	0.08	0.08	90
bus	0.12	0.16	0.13	90
motorcycle	0.14	0.19	0.16	90
pickup truck	0.02	0.01	0.01	90
train	0.30	0.36	0.33	90
lawn-mower	0.30	0.29	0.30	90
rocket	0.20	0.18	0.19	90
streetcar	0.10	0.13	0.11	90
tank	0.06	0.06	0.06	90
tractor	0.13	0.09	0.11	90
accuracy			0.18	9000
macro avg	0.17	0.18	0.17	9000
weighted avg	0.17	0.18	0.17	9000